DS5006 Machine Learning — Take-home Lab 5: Clustering

Fall 2026 · IIT Palakkad

1. What this lab is about

You will implement, from scratch and with NumPy only, the four clustering methods from the clustering lectures:

- **K-means** (Expectation and Maximization steps, the loss J),
- **K-medoids** (working from a dissimilarity matrix only),
- **Gaussian mixture models** fitted by **EM**, initialised from K-means,
- **agglomerative hierarchical clustering** with single, complete and average linkage, and cutting the dendrogram into K clusters.

This lab is about **implementation**. Most of the marks (85 of 100) are awarded automatically for code that follows the specification exactly. The four experiments in Part B are fully specified — there are no design decisions to make — and the report asks four short questions about what you see.

A student who has followed the lectures should need roughly 8–12 hours. Every function is short (the reference solution is about 300 lines in total). The difficulty lies in getting every detail of the specification right, so **read Section 3 carefully before writing code**.

2. Files

2.1 Files you are given

File	What it is	Edit it?
`ds5006_lab5.py`	The file you complete: 14 functions (Part A), 4 experiments (Part B). The docstring at the top lists all 14 graded conventions.	**Yes**
`clusterlab_provided.py`	Datasets, initial centres, a distance matrix helper, `matching_accuracy`, and the experiment constants.	**No** — replaced by a clean copy when graded
`test_lab5_public.py`	Self-check. Run `python test_lab5_public.py`.	No
`Lab5_Report_Template.md`	Skeleton for your report.	Copy it

All three Python files must sit in the same folder.

2.2 Files you upload

Upload **one zip file named with your roll number**, for example `CS24M001.zip`, containing exactly these three files at the top level (no sub-folder inside the zip):

File	Contents
`ds5006_lab5.py`	Your completed code, with `ROLL_NUMBER` set.
`results.json`	Written by running `python ds5006_lab5.py`.
`report.pdf`	Your report, at most **4 pages** including figures.

Do **not** upload `clusterlab_provided.py`, the test file, your plotting script, or any data. Submissions whose zip is not named by roll number, or whose `ds5006_lab5.py` is missing, are not graded automatically.

Your plotting code can live in a separate script (for example `plots.py` that imports your functions); it is not uploaded or graded.

3. Part A — the functions (65 marks)

Notation follows the lecture notes: N points $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^d$ stored as the rows of `X` (shape `(N, d)`), K clusters indexed $k = 0, \dots, K-1$ in code, and a clustering is a function C with $C(i) = k$ meaning "point i is in cluster k". In code, C is the integer array `labels` of shape `(N,)`.

General rules for every function

- NumPy only. No `sklearn`, `scipy`, or other learning libraries.
- Do not print, do not read files, do not use global variables.
- Return the types the docstring states. A scalar must be a Python `float` (use `float(...)`); labels and indices must be integer arrays.
- Do not modify the arrays you are given.

3.1 Distances and scatter

`pairwise_sq_dists(A, B)` returns the `(n_A, n_B)` matrix of **squared** Euclidean distances $\|\mathbf{a} - \mathbf{b}\|^2$. It must be vectorised (no Python loop over rows) and must never return a negative number — rounding can make an expansion like $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^\top\mathbf{b}$ slightly negative, so clip at zero. *(3 marks)*

`within_cluster_scatter(D, labels)` returns the within-cluster scatter

$$W(C) = \frac{1}{2} \sum_{k} \sum_{C(i)=k} \sum_{C(i')=k} d_{ii'}$$

using whatever dissimilarities are in `D`. *(3 marks)*

3.2 K-means

K-means alternates two steps.

- **E step** — fix the centres $\mathbf{m}_k$ and assign every point to the nearest one, $C(i) = \arg\min_k \|\mathbf{x}_i - \mathbf{m}_k\|^2$.
- **M step** — fix the assignment and move every centre to the mean of its points, $\mathbf{m}_k = \frac{1}{N_k} \sum_{C(i)=k} \mathbf{x}_i$.

The loss these steps reduce is

$$J = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}_{C(i)}\|^2.$$

Function	Specification	Marks
`kmeans_e_step(X, centres)`	Nearest centre. **Ties go to the smaller centre index.**	3
`kmeans_m_step(X, labels, centres)`	Mean of each cluster. **A cluster with no points keeps its previous centre.** Returns a new array.	4
`kmeans_loss(X, labels, centres)`	J as above: a **sum**, not a mean, and with no N_k factor.	2
`kmeans(X, init_centres, max_iter)`	The full algorithm, with the bookkeeping below.	7

Iteration bookkeeping (used by both `kmeans` and `kmedoids`). This is graded exactly.

```
labels  = E-step(initial centres);          record loss      # history[0]
repeat:
    centres = M-step(labels);               record loss      # after M
    new     = E-step(centres);              record loss      # after E
    if new is identical to labels: stop
    labels  = new
    (also stop after max_iter M steps)
n_iter = number of M steps performed
```

So the history has length $1 + 2\,n_{\text{iter}}$ and its last two entries are equal when the algorithm stops because the assignment did not change. Return `(labels, centres, history, n_iter)`.

Worked example (lecture notes). Points $1, 2, 6, 7, 9$, $K = 2$, initial centres 1 and 2. The history is $90,\ 26,\ 11,\ 5.1\overline{6},\ 5.1\overline{6}$, with $n_{\text{iter}} = 2$, final clusters $\{1, 2\}$, $\{6, 7, 9\}$ and centres 1.5, $7.\overline{3}$. The public self-check tests exactly this.

3.3 K-medoids

K-medoids sees **only a dissimilarity matrix** `D` — never `X`. Its centres are data points (medoids), stored as row indices.

- **E step** — assign each point to the medoid with the smallest dissimilarity, `argmin_k D[i, medoids[k]]`. Ties go to the smaller k.
- **M step** — for each cluster k, the new medoid is the member i of cluster k with the smallest total dissimilarity to the other members, $\sum_{C(i')=k} D_{ii'}$. **Ties go to the smallest row index.** If a cluster is empty, its medoid does not change.
- **Loss** — $\sum_i D[i,\ \text{medoid of the cluster of } i]$.

`kmedoids(D, init_medoids, max_iter)` uses exactly the same bookkeeping as `kmeans` and returns `(labels, medoids, history, n_iter)`, where `labels` are positions $0..K-1$ in `medoids`. *(7 marks)*

3.4 Gaussian mixture models and EM

The mixture density is $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

`gaussian_logpdf(X, mu, Sigma)` returns, for every row,

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\tfrac{1}{2}\Big[d \log 2\pi + \log|\boldsymbol{\Sigma}| + (\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big].$$

(4 marks)

`gmm_e_step(X, pis, mus, Sigmas)` returns the responsibilities

$$\gamma(z_{ik}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

as an `(N, K)` array, and the **total** log-likelihood $\sum_i \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ as a float. It must work for a point so far from every component that each density $\mathcal{N}(\cdot)$ is 0 in floating point. *Hint:* work with $a_{ik} = \log \pi_k + \log \mathcal{N}(\mathbf{x}_i \mid \cdot)$; subtracting $\max_k a_{ik}$ from row i before exponentiating does not change $\gamma(z_{ik})$ and keeps the numbers finite. *(6 marks)*

`gmm_m_step(X, R, reg)` with $N_k = \sum_i \gamma(z_{ik})$:

$$\pi_k = \frac{N_k}{N}, \qquad \boldsymbol{\mu}_k = \frac{1}{N_k}\sum_i \gamma(z_{ik})\,\mathbf{x}_i, \qquad \boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_i \gamma(z_{ik})\,(\mathbf{x}_i-\boldsymbol{\mu}_k)(\mathbf{x}_i-\boldsymbol{\mu}_k)^\top + \texttt{reg}\cdot\mathbf{I}.$$

The covariance is an **outer** product, uses the **new** mean, divides by N_k, and then adds `reg` to the diagonal. *(6 marks)*

`gmm_init_from_kmeans(X, labels, centres, reg)` — "run K-means first, then EM". With N_k the number of points in cluster k: $\pi_k = N_k/N$, $\boldsymbol{\mu}_k =$ `centres[k]`, and $\boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_{C(i)=k}(\mathbf{x}_i - \mathbf{m}_k)(\mathbf{x}_i - \mathbf{m}_k)^\top + \texttt{reg}\cdot\mathbf{I}$ where $\mathbf{m}_k$ is **the centre you are given** (`centres[k]`), not a recomputed mean. *(3 marks)*

`gmm_em(X, pis, mus, Sigmas, reg, tol, max_iter)` — graded exactly:

```
R, ll = E-step(initial parameters);     record ll          # history[0]
repeat:
    parameters = M-step(R)
    R, ll      = E-step(parameters);    record ll
    if history[-1] - history[-2] < tol: stop               # no abs()
    (also stop after max_iter M steps)
n_iter = number of M steps performed
```

Return `(pis, mus, Sigmas, R, history, n_iter)`; `R` comes from the **final** parameters and the history has length $1 + n_{\text{iter}}$. *(6 marks)*

3.5 Agglomerative hierarchical clustering

Start with every point as its own cluster and repeatedly merge the two closest clusters until one remains. The distance between clusters G and H is

- **single linkage:** $\min_{i\in G,\, i'\in H} d_{ii'}$,
- **complete linkage:** $\max_{i\in G,\, i'\in H} d_{ii'}$,
- **average linkage:** $\frac{1}{|G|\,|H|}\sum_{i\in G}\sum_{i'\in H} d_{ii'}$.

`agglomerative(D, linkage)` returns an `(N-1, 4)` float array. Row t records merge t as `[i, j, height, size]`:

- points have ids $0, \ldots, N-1$; the cluster created by merge t gets id $N+t$;
- `i < j` are the ids of the two clusters merged;
- `height` is their linkage distance;
- `size` is the number of points in the new cluster.

When several pairs are equally close, merge the pair with the **smallest `i`**, and among those the **smallest `j`**. *(8 marks)*

Worked example. For the points $1, 2, 6, 7, 9$ with $d_{ii'} = |x_i - x_{i'}|$ and single linkage the merges are `[0,1,1,2]`, `[2,3,1,2]` (a tie with the first, broken by `i`), `[4,6,2,3]` (point 9 joins cluster $6 = \{6, 7\}$), `[5,7,4,5]`.

You may recompute linkage distances from their definitions after every merge, or update them; both are accepted. Recomputing is simpler but slower; either way, the experiment with $N = 200$ must finish within a minute.

`cut_dendrogram(merges, N, K)` keeps only the first $N - K$ merges and returns the resulting K clusters as labels $0..K-1$, **numbered in order of first appearance**: point 0 is in cluster 0, the lowest-indexed point not in cluster 0 is in cluster 1, and so on. *(3 marks)*

4. Part B — the experiments (20 marks)

Each experiment is a function `experiment_n(roll)` returning a dictionary with exactly the keys in its docstring, built only from your Part A functions and the provided module. Everything is specified; follow it literally. Use the constants (`K_RANGE`, `N_RESTARTS`, `GMM_REG`, ...) by name. Running `python ds5006_lab5.py` calls all four and writes `results.json`.

Each dataset comes from `make_dataset(roll, name)`, which returns `(X, y)`. **`y` is used only for evaluation** (accuracy, shifts, plots) — never pass it to a clustering function. Every roll number gets a different dataset.

Experiment 1 — choosing K and restarts (dataset `"blobs"`, 5 marks). For every K in `K_RANGE` and every r in `range(N_RESTARTS)`: `idx = initial_centre_indices(roll, "blobs", K, r)`; run `kmeans(X, X[idx])`; take the final loss `history[-1]`. Return `best_J` and `worst_J`: the smallest and largest final loss over the restarts, one value per K, in the order of `K_RANGE`.

Experiment 2 — outliers (dataset `"outliers"`, 5 marks). `idx = initial_centre_indices(roll, "outliers", 2, 0)`. Run `kmeans(X, X[idx])`. Compute `D = euclidean_distance_matrix(X)` and run `kmedoids(D, idx)` from the same starting points. Let $\mathbf{g}_0, \mathbf{g}_1$ be the means of `X[y == 0]` and `X[y == 1]` (the two real groups). The *shift* of a point $\mathbf{p}$ is $\min(\|\mathbf{p} - \mathbf{g}_0\|, \|\mathbf{p} - \mathbf{g}_1\|)$. Return the two K-means centres, the two medoid indices, `kmeans_shift` (the larger shift of the two K-means centres) and `kmedoids_shift` (the larger shift of the two medoid points `X[medoids]`).

Experiment 3 — K-means then EM (dataset `"aniso"`, 5 marks). `idx = initial_centre_indices(roll, "aniso", 3, 0)`. Run `kmeans(X, X[idx])`, then `gmm_init_from_kmeans(X, labels, centres, GMM_REG)`, then `gmm_em` from those parameters with its default arguments. The GMM labels are `np.argmax(R, axis=1)`. Return `kmeans_acc = matching_accuracy(kmeans labels, y)`, `gmm_acc` likewise, `gmm_pis_sorted` (the three π_k, largest first), `gmm_initial_ll` (`history[0]`), `gmm_final_ll` (`history[-1]`) and `gmm_n_iter`.

Experiment 4 — linkage (dataset `"moons"`, 5 marks). `D = euclidean_distance_matrix(X)`. For each linkage in `LINKAGES`, run `agglomerative(D, linkage)` and `cut_dendrogram(merges, N, 2)`. Return, for each linkage, `acc` = `matching_accuracy(labels, y)` and `top_height` = the height of the last merge.

How Part B is graded. The grader calls your four experiment functions on your roll number and compares the output with the reference implementation (relative tolerance 10^{-6} on losses and log-likelihoods, 10^{-4} on coordinates and shifts, exact on indices). Because Part B is built on Part A, a bug in a Part A function usually costs marks in both. The grader also checks that your uploaded `results.json` matches what your code produces; a mismatch is flagged for manual review.

5. The report (15 marks)

At most **4 pages**, PDF, using the template. For each experiment: the figure(s) asked for and a short answer — **2 to 5 sentences**. Answers that restate numbers without explaining them earn little.

Q1 (Experiment 1, 4 marks). Plot `best_J` and `worst_J` against K on one set of axes. Which K would you choose for your data, and why? Why is "the K with the smallest loss" not a usable rule? Why are `best_J` and `worst_J` different for some values of K?

Q2 (Experiment 2, 3 marks). Scatter plot of the data with the two K-means centres and the two medoids marked. Explain why the K-means centre moved away from its group and the medoid did not.

Q3 (Experiment 3, 4 marks). Two scatter plots side by side, coloured by the K-means labels and by the GMM labels, and a plot of the EM log-likelihood history. What did K-means get wrong on this dataset, and which feature of the Gaussian mixture model corrected it? Compare `gmm_pis_sorted` with the true group sizes (count them from `y`).

Q4 (Experiment 4, 4 marks). Three scatter plots coloured by the two-cluster cut, one per linkage. Explain why single linkage recovers the two moons and complete linkage does not. Why is single linkage's `top_height` so much smaller than complete linkage's?

6. Marks

Component	How	Marks
Part A: 14 functions	Automatic, hidden tests	65
Part B: 4 experiments	Automatic, compared with reference on your roll	20
Report	By hand	15
Total		**100**

The grader runs your file in a fresh folder with a clean copy of `clusterlab_provided.py`, a time limit, and a static scan of your imports. A file that does not import (syntax error, banned import, missing function) scores **0** for Parts A and B. A function that raises an error or exceeds the time limit scores 0 for its tests and the others are still graded.

7. Before you submit — checklist

1. `ROLL_NUMBER` is set at the top of `ds5006_lab5.py`.
2. `python test_lab5_public.py` reports 20/20. (Passing these is necessary, not sufficient: the hidden tests are different and larger.)
3. `python ds5006_lab5.py` runs without error in under a minute and writes `results.json`.
4. Your file imports nothing except `json`, `numpy` and `clusterlab_provided` (and the standard library).

5. The zip is named `<ROLL>.zip` and contains exactly `ds5006_lab5.py`, `results.json` and `report.pdf`, with no folder inside.

Troubleshooting

Symptom	Likely cause
`ModuleNotFoundError: clusterlab_provided`	The three `.py` files are not in the same folder, or you are running Python from a different folder. `cd` into it first.
K-means history has the wrong length	Re-read the bookkeeping box: one E step before the loop, then M and E inside it.
`gmm_e_step` gives `nan`	You exponentiated before normalising. See the hint in Section 3.4.
GMM log-likelihood decreases	Covariance around the old mean, inner instead of outer product, or N instead of N_k.
`agglomerative` is very slow	A Python loop over all pairs of *points* inside the merge loop. Keep a matrix of *cluster* distances instead.

8. Academic integrity

The course policy applies in full: submitted work must be your own, and copying from other students, the internet or GenAI tools is not permitted. Discussing ideas is encouraged; sharing code is not. A short viva may be held for any submission, where you will be asked to explain and modify your own code.